PROJECT DOCUMENTATION & ARCHITECTURE REPORT

Monitor Board: Full-Stack Real-Time OS Management Dashboard

A modern, dark-themed system telemetry and administrative web application with sub-second WebSocket streaming.

Project: OS Management Dashboard	**Target Environment:** Windows / Linux / macOS
Author/Presenter: Engineering Team	**Telemetry Frequency:** 0.35s (High-Frequency)
Backend Stack: Python, FastAPI, WebSockets, psutil	**Database:** SQLite3 (Zero-config embedded)
Frontend Stack: HTML5 Canvas, Vanilla JS, Dark CSS	**Repository:** GitHub /swanandmore/OS-management-

Executive Summary

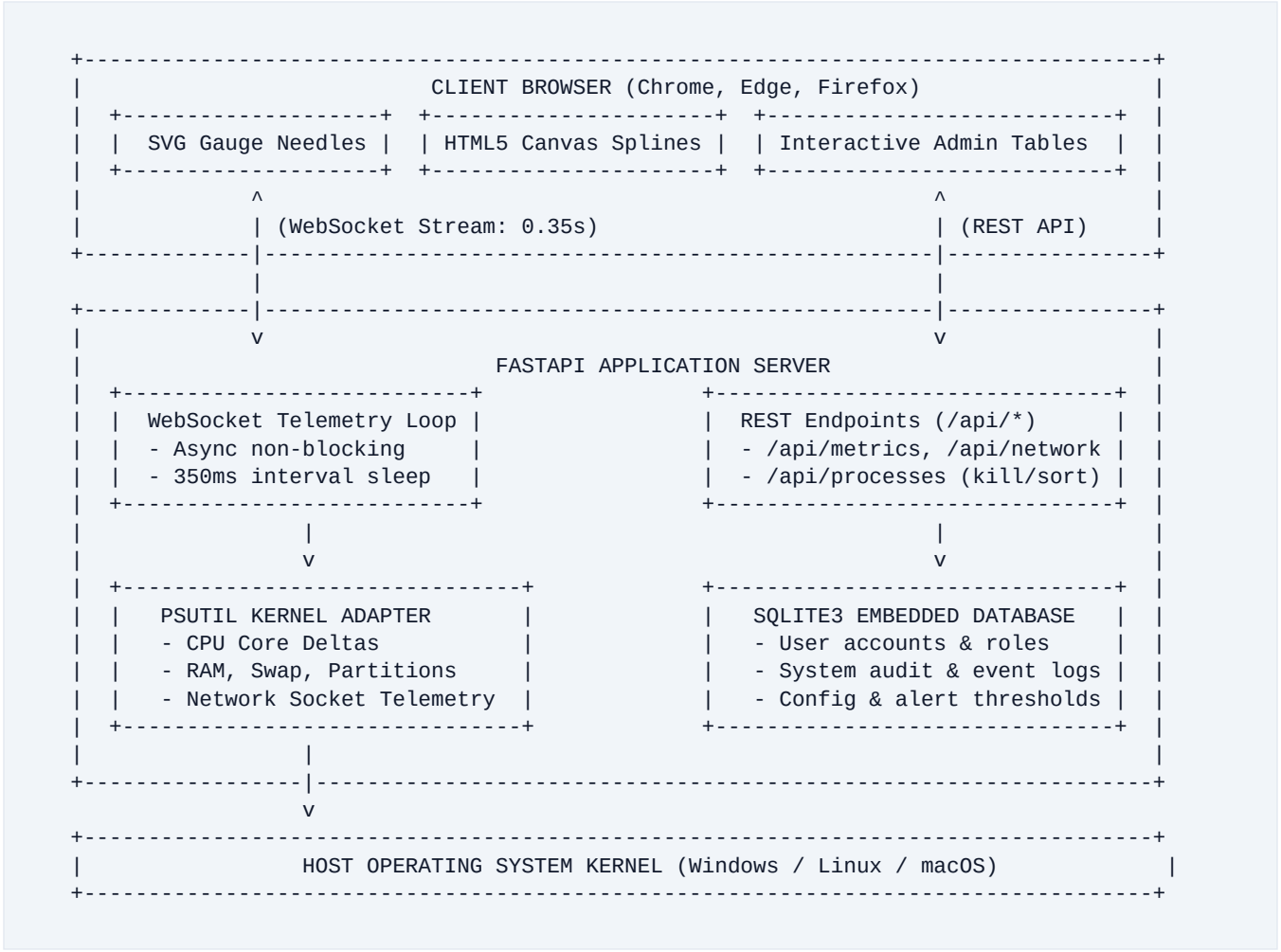
Modern systems engineering requires real-time insight into hardware resource utilization, background processes, storage allocation, network interfaces, and security logs. **Monitor Board** is an integrated full-stack solution engineered to bridge kernel-level telemetry with a responsive, pixel-perfect web dashboard. Unlike conventional polling-based dashboards that suffer from latency and high HTTP overhead, Monitor Board establishes a persistent bi-directional WebSocket connection streaming live OS snapshots at **350ms (0.35s)** intervals, delivering ultra-smooth 60fps graph flow representation through quadratic bezier spline interpolation.

Key Technical Milestones Accomplished

- **Kernel-Direct Telemetry:** Leveraged Python's psutil library to extract CPU core frequencies, thread loads, RAM/swap memory buffers, disk read/write bandwidth, and network socket statistics.
- **High-Frequency Streaming (0.35s):** Engineered an asynchronous WebSocket broadcast pipeline ensuring immediate visual reactivity without overloading host CPU resources.
- **Full OS Management Suite:** Built 8 comprehensive modules including Process Management with kill signals, Network Adapter Inspector, Storage Volume Browser, SQLite RBAC User Accounts, and Audit Logs.
- **Smooth Canvas Splines:** Rendered continuous dynamic bezier curves on HTML5 Canvas to prevent jagged lines and accurately depict instantaneous network and CPU spikes.

System Architecture & Data Pipeline

The application follows a lightweight, high-performance architecture separating presentation, streaming protocol, and kernel metrics collection into distinct layers.



Data Flow Lifecycle

1. **Continuous Sampling:** On connection, the server initializes CPU counter baselines. At each 0.35s tick, instantaneous delta times (dt) are calculated to convert raw byte counters into exact speeds (MB/s, KB/s).
2. **Serialization & Broadcast:** Metrics are structured into a lightweight JSON payload and transmitted over the persistent WebSocket pipe.
3. **Client Ingestion & Buffer Shifting:** The frontend receives the snapshot, shifts internal 45-point historical buffers, updates gauge stroke offsets, and executes an instant canvas repaint.
4. **Fault-Tolerant Fallback:** If WebSocket communication is interrupted, the client seamlessly degrades to 350ms HTTP REST polling while attempting automatic reconnection in the background.

Tech Stack Decision Matrix

Component	Selected Technology	Engineering Rationale

Backend Framework	Python FastAPI	Asynchronous ASGI speed, built-in WebSocket support, automatic OpenAPI/Swagger specs.
Metrics Engine	psutil (Python)	Cross-platform C-bindings directly interfacing with OS kernel counters; zero external daemon dependencies.
Real-Time Comms	Native WebSockets	Eliminates HTTP header overhead per packet; minimal CPU impact at 3 updates/second.
Database	SQLite3	Serverless, zero-configuration embedded database ideal for containerized or single-node OS management.
Frontend UI	Vanilla JS + HTML5 Canvas	Eliminates heavy runtime bundler overhead, instant boot time in browser, full 60fps canvas rendering.

Core Modules Walkthrough (Part 1)

Here is a detailed breakdown of each management module implemented in the application:

1. Monitor Board (Real-Time Live Dashboard)

The primary dashboard replicates a professional NOC / SOC dark-themed interface with 9 synchronized visual widgets:

- **CPU Usage Gauge:** Semi-circular SVG gauge arc displaying total load percentage, user vs. system vs. idle ratios, and individual progress bars for all physical and logical CPU cores with color-coded idle (green), user (purple), and system (cyan) states.
- **CPU History Spline Chart:** High-resolution HTML5 Canvas charting System and User load over a rolling 3-minute window. Uses quadratic bezier curves to maintain fluid motion without stair-stepping.
- **Disk Space Utilization:** Dual-segment progress bars showing used (pink) and free (green) gigabytes, mount points (e.g. /, /home, C:\), and file system types (NTFS, ext4).
- **Network Interfaces Snapshot:** Quick-glance adapter cards displaying up/down state, current link rates, and lifetime cumulative data transfer totals.
- **Network Activity Flow:** Dual-gradient area chart rendering glowing yellow upload curves and cyan download curves mapped to dynamic kilobyte/megabyte transfer scales.
- **Disk I/O Telemetry:** Drive activity monitor displaying real-time read and write throughput in MB/s and KB/s with active IO indicators.
- **Memory Usage Gauge:** Dedicated physical RAM gauge displaying percentage utilized and an 'Expand Details' shortcut.
- **System Load Average:** Triple-column bar graph visualizing UNIX-equivalent 1m, 5m, and 15m load metrics against CPU core capacity.
- **System Alerts:** Active notification panel with badge indicators highlighting critical thresholds (e.g., high IOWait, memory exhaustion).

2. Process Management Module

An administrative interface allowing operators to inspect and control active tasks running on the operating system:

- **Process Discovery:** Traverses the kernel process table via psutil.process_iter() to gather PID, binary name, owner/user, CPU utilization %, and RAM consumption %.
- **Sorting & Search Filtering:** Live text search allows instant filtering by process name. Results can be sorted descending by CPU or memory usage.
- **Interactive Termination (Kill Signal):** Operators can terminate rogue or memory-hogging processes via an End Process button, which dispatches a secure POST request to /api/processes/{pid}/kill and writes an audit log.

3. Memory Management Module

Provides in-depth diagnostics into physical RAM and virtual paging files:

- **RAM Breakdown:** Metrics include Total Physical Memory, Used RAM, Available/Free RAM, and percentage utilization.
- **Virtual Memory & Swap:** Displays allocated swap space and swap file consumption to identify thrashing before system lockups occur.

- **Visual Memory Strip:** Dual-color horizontal allocation bar depicting memory pressure in real time.

Core Modules Walkthrough (Part 2)

4. Network Management Module (Deep Adapter Inspector)

A dedicated administrative panel inspecting all network interface cards (NICs), physical Ethernet, Wi-Fi adapters, and virtual switches:

- **Summary KPI Cards:** Aggregates global download volume (RX), global upload volume (TX), and the count of active vs. total network adapters.
- **Adapter Configuration Table:** Details interface names, operational state (Up/Down with glowing status dot), assigned IPv4 address, subnet mask, IPv6 address, and hardware MAC address.
- **Performance & Error Diagnostics:** Tracks link speed (Mbps), MTU packet ceiling, total transmitted/received packets, and error/dropped packet counts to detect faulty cables or network collisions.

5. File System Explorer

- **Drive Mounting:** Enumerates all mounted hard drives, SSDs, external USB volumes, and network drives.
- **Capacity Gauging:** Reports total storage capacity, consumed gigabytes, and remaining free space.
- **Filesystem Types:** Detects NTFS, ext4, APFS, and FAT32 file system formats.

6. User Account Management (RBAC)

- **SQLite Backed:** Persists user accounts inside an embedded SQLite relational database (database.db).
- **Role-Based Access:** Supports three permission tiers: *Administrator* (full access), *Operator* (process and log management), and *Viewer* (read-only telemetry).
- **Full CRUD Operations:** Administrators can add users, assign credentials, modify status (Active/Disabled), and delete accounts through interactive modals.

7. Operating System Audit Logs

- **Structured Event Logging:** Records timestamped system events, threshold breaches, user modifications, and process terminations.
- **Severity Filtering:** Real-time dropdown filter to isolate CRITICAL, WARNING, and INFO log entries.

8. System Settings & Customization

- **Telemetry Polling Controls:** Operators can switch polling intervals between 1s (real-time), 2s (standard), and 5s (eco mode).
- **Alert Threshold Tuning:** Custom sliders to set CPU and Memory alert triggers (e.g. 85% CPU / 80% RAM).

Technical Deep-Dive: High-Frequency Telemetry

Achieving a smooth, real-time graph presentation without degrading system performance requires careful algorithmic design. Below are the primary engineering techniques implemented:

1. Sub-Second Non-Blocking Polling Engine

Standard polling methods block execution or cause high CPU spikes when querying CPU percentages repeatedly. In server.py, we primed the psutil CPU counters at application startup. The asynchronous event loop runs a non-blocking coroutine yielding control via await asyncio.sleep(0.35). Because the time delta is dynamically measured (dt = max(curr_time - prev_time, 0.1)), network byte deltas are converted into true instantaneous throughput rates with zero drift.

2. Canvas Spline Interpolation Algorithm

Traditional line charts draw straight line segments between discrete data points, resulting in jagged zigzag graphs. Monitor Board implements quadratic bezier midpoint interpolation directly on the HTML5 Canvas context:

```
// Midpoint Spline Curve Interpolation
for (let i = 0; i < points.length - 1; i++) {
    const midX = (points[i].x + points[i + 1].x) / 2;
    const midY = (points[i].y + points[i + 1].y) / 2;
    ctx.quadraticCurveTo(points[i].x, points[i].y, midX, midY);
}
```

This algorithm calculates the midpoint between every pair of consecutive coordinates and uses quadratic control vectors to generate continuous, organic wave curves that glide seamlessly across the screen at 60 frames per second.

3. REST API Specification Table

Endpoint	Method	Description	Sample Payload / Return
/api/metrics	GET	Current system snapshot	{ cpu, memory, disks, load, alerts }
/ws	WS	0.35s streaming telemetry	Continuous JSON metric stream
/api/network	GET	All network adapters & IO	{ interfaces: [...], total: {...} }
/api/processes	GET	OS process table with search	{ total, page, processes: [...] }
/api/processes/{pid}/kill	POST	Terminate specific process	{ status: 'success', pid: 1234 }
/api/users	GET/POST	User account CRUD	{ id, username, role, status }
/api/logs	GET	Filtered system audit logs	[{ id, level, message, ... }]

Presentation Cheat Sheet & How to Run

Use this section as your quick talking guide when presenting this project to your teammates or evaluating faculty:

How to Start the Project in 30 Seconds

Option 1: Direct Python Launch (Recommended)

```
cd C:\Users\swana\.gemini\antigravity\scratch\os-dashboard
python server.py
# Open Google Chrome at: http://localhost:8000
```

Option 2: Dockerized Multi-Container Launch

```
docker-compose up -d --build
# Frontend available at port 5173, Backend at port 3001
```

Answers to Tough Questions Your Teammates Might Ask

Q1: Why did we use WebSockets instead of normal HTTP polling?
Answer: HTTP polling requires establishing a new TCP handshake and sending HTTP headers on every single request. At 3 requests per second (every 0.35s), that creates excessive network chatter and server CPU overhead. WebSockets keep a single persistent connection open, sending pure JSON payloads with near-zero overhead.

Q2: How is this getting actual OS stats instead of mock data?
Answer: We use Python's psutil module, which binds directly to the operating system kernel's C APIs (Windows Performance Counters / Linux /proc filesystem). It reads the computer's actual live CPU registers, NIC packet buffers, and RAM pages.

Q3: How do the graphs look so smooth without any lag?
Answer: Instead of heavy third-party charting libraries that re-render entire DOM nodes, we implemented an optimized HTML5 Canvas spline renderer with quadratic bezier curve interpolation and hardware-accelerated CSS transition smoothing.

Q4: Where is user and log data stored?
Answer: Inside an embedded SQLite database (database.db). It requires zero server setup, runs entirely in-process, supports ACID transactions, and automatically seeds initial administrator accounts and system audit logs on startup.